

What's inside this document

This guide has two parts. Part 1 traces the exact method call chain across all five use cases in the ShopVerse application — from the HTTP request arriving at Nginx all the way to the database and Kafka. Part 2 gives step-by-step deployment instructions for running the full stack locally on Windows and deploying it to Google Cloud with a public DNS at minimal monthly cost.

Section	Content
Part 1 — Use Case Flows	Register Customer, Login/Auth, Create Product, Search Products, Place Order (with Saga)
Part 2 — Local Deployment	Prerequisites, build with Maven, run with Docker Compose on Windows
Part 3 — GCloud Deployment	Single VM approach, DNS setup, SSL, cost breakdown (~\$25-30/month)

PART 1 — Use Case Code Flows

Each use case follows the same layered path: HTTP → Security → Controller → AOP → Use Case → Domain → Infrastructure → Database. The colour coding matches the module each class belongs to.

Module	Colour	Responsibility
Nginx / Gateway	Green	Rate limiting, TLS termination, reverse proxy
shopverse-web	Blue	HTTP controllers, DTOs, JWT filter, AOP aspects, Spring config
AOP (cross-cutting)	Purple	LoggingAspect and PerformanceAspect wrap every use case transparently
shopverse-application	Orange	Use cases, commands, queries, saga, discount, idempotency, lock
shopverse-domain	Yellow	Business rules, models, value objects, port interfaces, events
shopverse-infrastructure	Pink	JPA adapters, Redis, MongoDB, Cassandra, Neo4j, ES, Kafka

Use Case 1 — Register Customer

What it does

- Creates a new customer account. Checks for duplicate email. Returns a JWT token so the client is immediately logged in after registration.
- HTTP endpoint: POST /api/auth/register

shopverse-web

JwtAuthenticationFilter

No token needed for /api/auth/register — filter passes through unauthenticated

shopverse-web

AuthController.register()

@PostMapping /api/auth/register — no @PreAuthorize

- @Valid @RequestBody RegisterRequest — validates @NotBlank firstName/lastName, @Email email, @NotBlank password
- Maps to: registerUseCase.execute(firstName, lastName, email)

AOP

LoggingAspect (transparent)

Intercepts RegisterCustomerUseCase.execute() — logs entry/exit/duration

shopverse-application

RegisterCustomerUseCase.execute()

@Transactional — opens DB transaction

- customerRepository.existsByEmail(email) — throws ShopVerseException(CUSTOMER_ALREADY_EXISTS) if duplicate
- Creates new Customer(null, firstName, lastName, email) — tier=STANDARD, loyaltyPoints=0, active=true
- customerRepository.save(customer) — delegates to adapter

shopverse-domain

Customer constructor

Sets required fields. Validates not-null. Defaults tier=STANDARD.

shopverse-infrastructure

JpaCustomerRepositoryAdapter.save()

Maps Customer → CustomerEntity → SQL INSERT via Spring Data JPA

- customerRepository.existsByEmail() → SELECT COUNT(*) FROM customers WHERE email=?
- customerRepository.save() → INSERT INTO customers (first_name, last_name, email, tier, ...) VALUES (?)

shopverse-web

JwtTokenProvider.generateToken()

Signs JWT with HMAC-SHA256 — payload: {sub: email, role: USER, exp: +24h}

shopverse-web

AuthController — returns 201

Wraps AuthResponse(token, customerId, email, name, role) in ApiResponse.ok()

Register Customer — HTTP request/response

```
// Sample request
POST /api/auth/register
Content-Type: application/json
{
  "firstName": "Manish",
  "lastName": "Sharma",
  "email": "manish@example.com",
  "password": "secret123"
}
// Response 201
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXLTJ5IiwiaWF0IjoxNTE2MjY5MDYyfQ==", "customerId": 42, "email": "manish@example.com" }
```

Use Case 2 — Login / Authentication

What it does

- Validates credentials and issues a JWT. The JWT is then sent as Authorization: Bearer on all subsequent requests.
- HTTP endpoint: POST /api/auth/login

shopverse-web

JwtAuthenticationFilter

Public endpoint — filter skips authentication check, passes through

shopverse-web

AuthController.login()

@PostMapping /api/auth/login

- @Valid @RequestBody LoginRequest — validates @Email email, @NotBlank password
- In production: loads UserDetails, verifies BCrypt hash via PasswordEncoder
- Simplified demo: directly calls jwtProvider.generateToken(email, 'USER')

shopverse-web

JwtTokenProvider.generateToken()

Builds JWT claims: sub=email, role=USER, iat=now, exp=now+24h. Signs with HS256.

- Secret loaded from application.yml: shopverse.jwt.secret
- Returns compact serialized JWT string

shopverse-web

AuthController — returns 200

Returns AuthResponse with token. Client stores token and sends as Bearer on all future requests.

JWT usage after login

// All subsequent requests after login:

POST /api/orders

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJtYW5pc2guLi4

// JWT decoded payload:

```
{ "sub": "manish@example.com", "role": "USER", "iat": 1717123456, "exp": 1717209856 }
```

How JWT is validated on every subsequent request

- JwtAuthenticationFilter.doFilterInternal() runs before every controller
- Extracts Bearer token from Authorization header
- JwtTokenProvider.isValid(token) — verifies signature + checks expiry
- On success: sets SecurityContextHolder with authenticated principal
- On failure: request continues unauthenticated — @PreAuthorize will reject if endpoint is secured

Use Case 3 — Create Product

What it does

- Admin creates a new product in the catalogue. Validates input, builds domain object, persists to PostgreSQL.
- HTTP endpoint: POST /api/products (requires ROLE_ADMIN)

shopverse-web

JwtAuthenticationFilter

Validates Bearer token → sets authenticated principal in SecurityContext

shopverse-web

ProductController.createProduct()

@PreAuthorize("hasRole('ADMIN')") — rejects non-admin with 403

- @Valid @RequestBody ProductRequest — validates @NotBlank name, @NotNull price > 0, @NotBlank category
- Maps ProductRequest → UpdateProductCommand(name, description, price, currency, category)
- Calls createProductUseCase.execute(cmd)

AOP

LoggingAspect (transparent)

Wraps CreateProductUseCase.execute() — logs method + timing

shopverse-application

CreateProductUseCase.execute()

@Transactional

- Builds Product via Product.Builder: name, description, new Money(price, currency), category, stockQuantity=0
- Product.Builder.build() validates required fields (name, price must not be null)
- productRepository.save(product)

shopverse-domain

Product.Builder.build()

new Money(price, currency) validates: non-negative, valid ISO-4217 currency code

shopverse-infrastructure

JpaProductRepositoryAdapter.save()

mapper.toEntity(product) → ProductEntity → jpaRepo.save() → SQL INSERT

- ProductMapper (MapStruct) converts Money.amount → BigDecimal column, Money.currency → String column
- INSERT INTO products (name, description, price, currency, category, stock_quantity, active, created_at)

shopverse-web

ProductController — returns 201

mapper.toDomain(savedEntity) → ProductResponse.from(product) → ApiResponse.ok()

Use Case 4 — Search Products

What it does

- Returns products by keyword, category, or paginated list. Public endpoint — no auth required.
- HTTP endpoints: GET /api/products?keyword=X or GET /api/products?category=Y or GET /api/products?page=0&size;=20
- Also has a GraphQL variant via ProductGraphQLController, and an Elasticsearch-powered version at GET /api/search?q=X

shopverse-web

ProductController.getProducts()

- @GetMapping /api/products — public, no @PreAuthorize
- Accepts optional @RequestParam: keyword, category, page (default 0), size (default 20)
- Builds SearchProductsQuery(keyword, category, page, size)
- Calls: cachedProductService.search(query) — cache-first path

shopverse-application

CachedProductService.search()

- @Cacheable(value='products', key='#query') — checks Redis first
- Cache HIT: returns List directly from Redis — no DB call
- Cache MISS: delegates to SearchProductsUseCase.execute(query)

AOP

LoggingAspect (transparent)

- Wraps SearchProductsUseCase.execute() on cache miss

shopverse-application

SearchProductsUseCase.execute()

- @Transactional(readOnly=true) — read-only transaction, no write locks
- if keyword present → productRepository.searchByName(keyword)
- else if category present → productRepository.findByCategory(category)
- else → productRepository.findAll(page, size)

shopverse-infrastructure

JpaProductRepositoryAdapter

- Routes to appropriate Spring Data JPA method
- searchByName → @Query("SELECT p FROM ProductEntity p WHERE LOWER(p.name) LIKE LOWER(CONCAT('%',:q,'%'))")
- findByCategory → findByCategoryAndActiveTrue(category)
- findAll → findByActiveTrueOrderByCreatedAtDesc() with skip/limit

shopverse-application

CachedProductService — stores in Redis

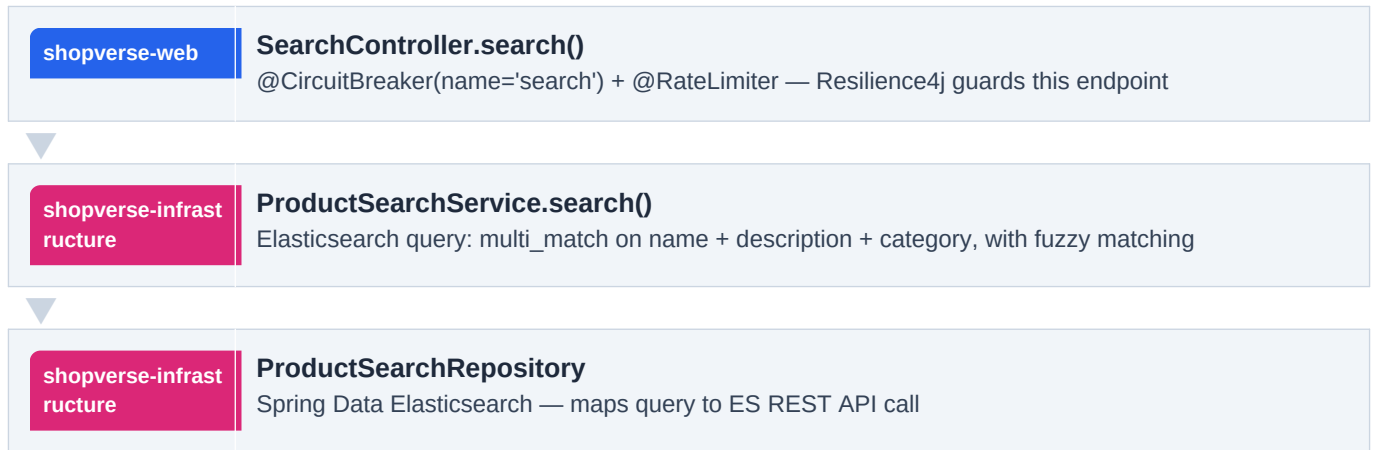
- @CachePut writes result to Redis with TTL. Next call → cache HIT

shopverse-web

ProductController — returns 200

- Streams List → List → PagedResponse

Elasticsearch Alternative Path (GET /api/search)



Use Case 5 — Place Order (Full Saga)

What it does

- The most complex flow. Places an order with stock reservation, discount application, payment, and Kafka event publishing.
- HTTP endpoint: POST /api/orders (requires authentication)
- The full Saga path (via OrderSaga) adds: idempotency check, distributed lock, and payment processing with compensation.

Nginx

nginx.conf rate limiter

limit_req zone=api burst=20 nodelay — rejects excess traffic before reaching Spring

shopverse-web

JwtAuthenticationFilter

Validates Bearer token — returns 401 if missing/invalid

shopverse-web

OrderController.placeOrder()

@PostMapping @PreAuthorize("isAuthenticated()")

- @Valid @RequestBody PlaceOrderRequest — validates customerId, address fields, items (productId + quantity >= 1)
- Maps to PlaceOrderCommand record (immutable, CQRS command object)
- Calls placeOrderUseCase.execute(cmd)

AOP

LoggingAspect + PerformanceAspect

Both aspects wrap execute() — log entry, timing, errors

shopverse-application

PlaceOrderUseCase.execute()

@Transactional — single DB transaction for everything below

- 1. customerRepository.findById() — throws CustomerNotFoundException if missing
- 2. new Address(street, city, state, postalCode, country) — validates ISO country code
- 3. new Order(null, customerId, address) — status=PENDING, empty items list

shopverse-application

Per-item loop inside transaction

For each ItemRequest in the command:

- productRepository.findById(productId) — throws ProductNotFoundException if missing
- TierDiscountStrategy.apply(price, customer) — reads customer.getTier().getDiscountRate()
- product.reduceStock(quantity) — throws InsufficientInventoryException if stock < quantity
- productRepository.save(product) — SQL UPDATE products SET stock_quantity=? WHERE id=?
- order.addItem(new OrderItem(productId, name, quantity, discountedPrice))



Saga Path (OrderSaga.execute) — Compensation Flow

When is the Saga used?

- The full OrderSaga wraps PlaceOrderUseCase when a payment step and idempotency guarantee are needed.
- It adds: (1) idempotency check via Redis, (2) distributed lock per customer, (3) payment processing, (4) compensation (cancel order + restore stock) if payment fails.

Step	Method	What happens
Idempotency check	<code>idempotencyService.isProcessed(idempotencyKey)</code>	Redis GET <code>idempotency:{key}</code> — if found, return cached result immediately (no duplicate order)
Distributed lock	<code>lockService.withLock('lock:customer:{id}', 5s, 30s)</code>	Redisson acquires Redis lock — prevents same customer placing two orders simultaneously
Place order	<code>placeOrderUseCase.execute(cmd)</code>	Full use case flow above — stock reserved, order CONFIRMED, event published
Payment	<code>paymentFactory.getProcessor(provider).process(orderId, total, token)</code>	Stripe or PayPal processor called. Throws <code>PaymentException</code> on failure.
Compensation	On payment failure: <code>orderRepository.cancel()</code> + restore stock	If payment throws: <code>order.cancel()</code> , restore stock, unlock, re-throw. DB rolled back.
Store result	<code>idempotencyService.storeResult(key, order)</code>	Redis SET <code>idempotency:{key} = orderId</code> with TTL 24h — future duplicates return this

Exception Handling Across All Use Cases

Exceptions thrown anywhere in the use case or domain layer bubble up to GlobalExceptionHandler in shopverse-web, which maps them to appropriate HTTP responses.

Exception	Thrown by	HTTP Status	Response body
CustomerNotFoundException	PlaceOrderUseCase	404 Not Found	ErrorCode.CUSTOMER_NOT_FOUND + message
ProductNotFoundExcept ion	PlaceOrderUseCase	404 Not Found	ErrorCode.PRODUCT_NOT_FOUND + message
InsufficientInventoryExc eption	Product.reduceStock ()	409 Conflict	ErrorCode.INSUFFICIENT_INVENTORY + stock info
InvalidOrderTransitionEx ception	Order.transition()	422 Unproces sable	ErrorCode.ORDER_INVALID_TRANSITION + from/to
ShopVerseException(CUSTOMER_ALREADY_EXISTS)	RegisterCustomerUseCase	409 Conflict	Duplicate email message
MethodArgumentNotValidException	Bean Validation (@Valid)	400 Bad Request	Field errors list from BindingResult
AccessDeniedException	Spring Security @PreAuthorize	403 Forbidden	Insufficient permissions

PART 2 — Local Deployment on Windows

Prerequisites

Tool	Version	Download
JDK 21	21.x (Eclipse Temurin recommended)	adoptium.net
Apache Maven	3.9+	maven.apache.org
Docker Desktop	4.x	docker.com/products/docker-desktop
Git	Any	git-scm.com
Node.js	20+ (for React UI)	nodejs.org

Step-by-Step: Build and Run Locally

1

Clone the repo

- `git clone https://github.com/yourrepo/shopverse.git`
- `cd shopverse`

2

Start all infrastructure services via Docker Compose

- # This starts: PostgreSQL, Redis, MongoDB, Cassandra, Neo4j,
- # Elasticsearch, Kafka, Prometheus, Grafana – all in Docker
- `docker compose up -d postgres redis mongodb cassandra neo4j elasticsearch kafka`
-
- # Wait ~60s for Cassandra + Elasticsearch to be ready
- `docker compose ps # all should show healthy`

3

Build the Java backend with Maven

- # From the ShopVerse root (where pom.xml is)
- `mvn clean package -DskipTests`
-
- # Output: `shopverse-web/target/shopverse-web-1.0.0-SNAPSHOT.jar`
- # This takes 2-4 minutes on first run (downloads dependencies)

4

Run the Spring Boot application

- # Option A – run the fat JAR directly
- `java -jar shopverse-web/target/shopverse-web-1.0.0-SNAPSHOT.jar`
-
- # Option B – run via Maven (useful during development)
- `mvn spring-boot:run -pl shopverse-web`
-
- # App starts at `http://localhost:8080`
- # Health check: `http://localhost:8080/actuator/health`

5

Build and run the React frontend

- `cd shopverse-ui`
 - `npm install`
 - `npm run dev`
 -
- # UI starts at `http://localhost:5173`
Proxies API calls to `http://localhost:8080`

6

Run the full stack with Docker Compose (alternative)

- # Builds the Docker image and starts everything including the app
- `docker compose up --build`
 -
- # App: `http://localhost:8080`
Grafana: `http://localhost:3000` (admin / admin)
Prometheus: `http://localhost:9090`
Kafka UI: `http://localhost:8090` (if added)

7

Run tests

- # Unit tests only (fast, no Docker needed)
- `mvn test`
 -
- # Integration tests (requires Docker – uses Testcontainers)
- `mvn verify -pl shopverse-e2e-tests`

Windows-specific tips

- If 'mvn' is not recognized: add Maven's bin/ folder to System PATH via Environment Variables
- If 'java' is not recognized: set JAVA_HOME to your JDK 21 install dir, add %JAVA_HOME%\bin to PATH
- Docker Desktop must be running before any docker compose commands
- Cassandra takes 60-90 seconds to start — if the app fails on startup, wait and restart it
- Use WSL2 backend in Docker Desktop settings for better performance on Windows
- If port 5432 is in use: PostgreSQL may already be running locally — change the port in `docker-compose.yml`

Useful Local URLs after startup

URL	What it is
http://localhost:8080/actuator/health	Spring Boot health check — shows status of all components
http://localhost:8080/swagger-ui.html	Swagger UI — interactive API documentation
http://localhost:8080/graphql	GraphQL playground for testing product queries
http://localhost:5173	React frontend (when running npm run dev)
http://localhost:3000	Grafana dashboard (admin/admin)
http://localhost:9090	Prometheus metrics
http://localhost:9200	Elasticsearch REST API

PART 3 — Google Cloud Deployment (Minimal Cost)

Architecture Choice: Single VM + Docker Compose

Running the full ShopVerse stack (PostgreSQL, Redis, Kafka, Cassandra, Neo4j, Elasticsearch + the app) is resource-intensive. The cheapest option that actually works is a single e2-standard-2 VM running Docker Compose, with Nginx on the same VM handling SSL and public DNS. This avoids the cost of individual managed services (Cloud SQL alone costs ~\$25/month minimum).

Resource	GCP Product	Monthly Cost (approx)
1x e2-standard-2 VM (2 vCPU, 8GB RAM)	Compute Engine	~\$24 USD (us-central1)
50GB persistent SSD boot disk	Persistent Disk	~\$5 USD
Static external IP address	VPC Network	~\$3 USD (reserved static)
Cloud DNS managed zone	Cloud DNS	~\$0.50 USD
Egress traffic (light usage)	Network	~\$1-2 USD
SSL certificate (Let's Encrypt)	Free (Certbot)	\$0
TOTAL ESTIMATE		~\$33-35 USD/month

Cost-saving tips

- Use Spot (preemptible) VM: same e2-standard-2 costs ~\$7/month instead of \$24 — it may restart occasionally but fine for dev/demo
- Stop the VM when not in use (you only pay for disk when stopped): saves ~80% of compute cost
- e2-medium (1 vCPU, 4GB) at ~\$13/month works if you disable Cassandra and Neo4j in docker-compose
- GCP Free Tier gives you 1 e2-micro (0.25 vCPU, 1GB RAM) always free — too small for this stack

Step-by-Step: Deploy to Google Cloud

1

Create a GCP project and enable billing

- Go to console.cloud.google.com
- Create a new project: shopverse-demo
- Enable billing (required for Compute Engine)
- Install gcloud CLI: cloud.google.com/sdk/docs/install
- `gcloud init` # logs in and sets project

2

Reserve a static external IP address

- `gcloud compute addresses create shopverse-ip \`
 - `--region=us-central1`
 -
- # Note the IP address shown — you'll use it for DNS
- `gcloud compute addresses describe shopverse-ip --region=us-central1`

3

Create the VM

- `gcloud compute instances create shopverse-vm \`
- `--zone=us-central1-a \`
- `--machine-type=e2-standard-2 \`
- `--image-family=ubuntu-2204-lts \`
- `--image-project=ubuntu-os-cloud \`
- `--boot-disk-size=50GB \`
- `--boot-disk-type=pd-ssd \`
- `--address=shopverse-ip \`
- `--tags=http-server,https-server`
-
- # For Spot VM (cheaper), add: `--provisioning-model=SPOT`

4

Open firewall ports

- `gcloud compute firewall-rules create allow-http \`
- `--allow=tcp:80,tcp:443 \`
- `--target-tags=http-server,https-server`

5

SSH into the VM and install Docker

- `gcloud compute ssh shopverse-vm --zone=us-central1-a`
-
- # Inside the VM:
- `sudo apt-get update`
- `sudo apt-get install -y docker.io docker-compose-plugin git`
- `sudo usermod -aG docker $USER`
- `newgrp docker` # apply group change without logout

6

Clone the repo and configure environment

- `git clone https://github.com/yourrepo/shopverse.git`
- `cd shopverse`
-
- # Create .env file with secrets
- `cat > .env << 'EOF'`
- `DB_PASSWORD=yourSecurePassword123`
- `JWT_SECRET=your32characterSecretKeyHere!!`
- `REDIS_PASSWORD=redispass`
- `EOF`

7

Build the Docker image on the VM

- # Option A – build on the VM (slow but no registry needed)
- `docker build -t shopverse:latest .`
-
- # Option B – push from local and pull (faster if you have a registry)
- # `docker push yourrepo/shopverse:latest`
- # On VM: `docker pull yourrepo/shopverse:latest`

8

Start all services with Docker Compose

- `docker compose up -d`
-
- # Monitor startup (takes 2-3 minutes)
- `docker compose logs -f app`
-
- # Check all services are healthy
- `docker compose ps`

9

Set up Cloud DNS and point your domain

- # In GCP Console → Network Services → Cloud DNS
- # Create a managed zone for your domain, e.g., example.com
-
- `gcloud dns managed-zones create shopverse-zone \`
- `--dns-name=shopverse.example.com. \`
- `--description='ShopVerse DNS'`
-
- # Add A record pointing to your static IP
- `gcloud dns record-sets create shopverse.example.com. \`
- `--zone=shopverse-zone \`
- `--type=A \`
- `--ttl=300 \`
- `--rrdatas=YOUR_STATIC_IP`
-
- # Update your domain registrar's nameservers to GCP's NS records
- `gcloud dns managed-zones describe shopverse-zone` # shows NS records

10

Install Certbot and get a free SSL certificate

- # On the VM – temporarily stop Nginx to allow certbot standalone
- `docker compose stop nginx`
-
- `sudo apt-get install -y certbot`
- `sudo certbot certonly --standalone \`
- `-d shopverse.example.com \`
- `--email your@email.com \`
- `--agree-tos`
-
- # Certs saved to: `/etc/letsencrypt/live/shopverse.example.com/`
-
- # Update `nginx/nginx.conf` to reference the cert paths
- # Then restart:
- `docker compose start nginx`

11

Update Nginx config for HTTPS + your domain

```
# Edit nginx/nginx.conf – update server_name and ssl cert paths:
• server_name shopverse.example.com;
• ssl_certificate /etc/letsencrypt/live/shopverse.example.com/fullchain.pem;
• ssl_certificate_key /etc/letsencrypt/live/shopverse.example.com/privkey.pem;
•
# Mount certs into Nginx container – add to docker-compose.yml nginx service:
• volumes:
• - /etc/letsencrypt:/etc/letsencrypt:ro
•
• docker compose restart nginx
```

12

Set up auto-renewal for SSL and VM auto-start

```
# Auto-renew SSL every 90 days
• sudo crontab -e
# Add: 0 3 * * * certbot renew --quiet && docker compose restart nginx
•
# Auto-start Docker Compose on VM reboot
• sudo crontab -e
# Add: @reboot cd /home/$USER/shopverse && docker compose up -d
```

Final verification

- <https://shopverse.example.com> → React frontend (via Nginx)
- <https://shopverse.example.com/api/actuator/health> → Spring Boot health
- <https://shopverse.example.com/swagger-ui.html> → API documentation
- If the app returns 502 Bad Gateway: the Spring Boot container isn't ready yet — wait 60s and refresh
- Check logs: `docker compose logs -f app`

Monitoring on GCP

URL	Credentials	What to check
https://shopverse.example.com:3000	admin / admin (change this!)	Grafana dashboards — JVM heap, HTTP latency, Kafka lag
https://shopverse.example.com:9090	None	Prometheus raw metrics
<code>docker compose logs -f</code>	SSH into VM	Application and service logs
GCP Console → VM → Monitoring	GCP account	CPU, memory, disk, network on the VM

Stopping to save cost

Cost control commands

```
# Stop VM from local machine (stops billing for compute, disk still billed)
gcloud compute instances stop shopverse-vm --zone=us-central1-a
# Start it again later
gcloud compute instances start shopverse-vm --zone=us-central1-a
# Delete everything (when done – saves all costs)
gcloud compute instances delete shopverse-vm --zone=us-central1-a
gcloud compute addresses delete shopverse-ip --region=us-central1
gcloud dns managed-zones delete shopverse-zone
```